

Lecture 3 • Mon 10/5/2026

CS160 Compilers — Fall 2026

Variables and Control Flow.

Yu Feng

Opening puzzle

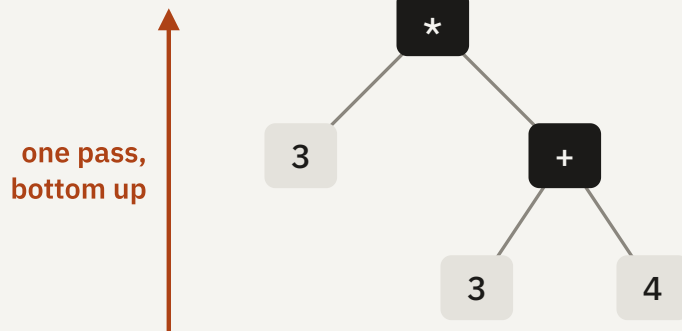
Wednesday's compiler meets a variable.

```
x: int = 3  
print(x * (x + 4))
```

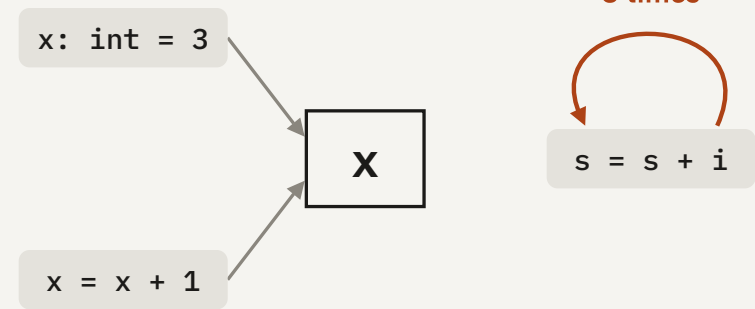
Wednesday's `compile_expr` has no case for `x`. Think for a minute, then talk it over with the person next to you: when we compile `x`, what *should* we emit? **Where does the 3 live?**

Wednesday asked you one of these. Here is the other.

Wednesday



Today



Every value was born, used once, and thrown away. The tree told you exactly when — a temporary is never needed again after the instruction above it.

The shape of the code *was* the order of execution. Read the program top to bottom and you have read exactly what runs, exactly once.

A variable breaks the first. A loop breaks the second. That's the whole lecture.

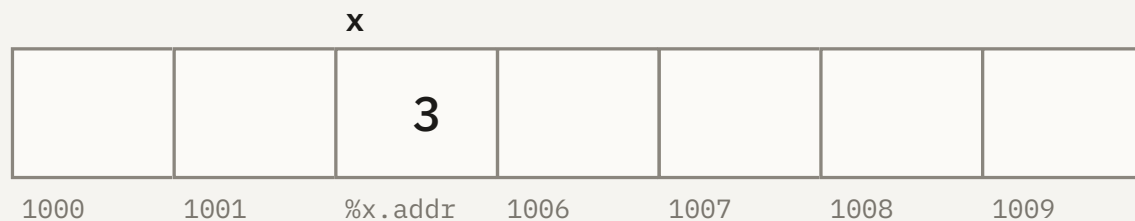
Where does a value live, and how does the program choose what to do next?

By the end you can compile a variable — because you'll know a name is really a *place*.

You can compile `if` and `while` — because you'll know there is no `if`, only labels and jumps.

PA1 needs both, and it's due Friday.

A variable is a name for a place.



Declaring `x` reserves a place — a **slot**: four bytes of Wednesday's memory strip, claimed for this program's use.

Reading `x` fetches whatever is there now.

Assigning `x = e` overwrites it.

Answer to the opening puzzle: we emit a *fetch*. Which means the compiler must have remembered where the place is.

alloca, load, store.

```
%x.addr = alloca i32          ; reserve a slot; %x.addr is its address
store i32 3, ptr %x.addr      ; put 3 there
%t1 = load i32, ptr %x.addr    ; fetch what's there into a temporary
```

- ptr** a **pointer** — a value that *is* an address, exactly as in C (Wednesday, slide 9)
- alloca** claim a slot and hand back its address — this is what `int x;` does in C. Where the space comes from is Lecture 8.
- store** note the order — value first, address second

The compiler has to remember where **x** is.

```
class Emitter:
    def __init__(self):
        self.lines = []
        self.n = 0                # counter for temporaries (Wednesday)
        self.env = {}            # NEW: variable name -> its slot
```

after `x: int = 3`

```
env = { "x": "%x.addr" }
```

environment — the compiler's dictionary from each variable's name to its slot. Every compiler has one; in week 5 it grows types and gets its real name, the *symbol table*.

One new line in the Emitter. Everything else about variables follows from it.

Declaration: reserve a slot.

```
case ast.AnnAssign(target=ast.Name(id=x), value=v):    # x: int = literal
    slot = "%" + x + ".addr"
    out.env[x] = slot
    out.emit(f"{slot} = alloca i32")
    val = compile_expr(v, out)
    out.emit(f"store i32 {val}, ptr {slot}")
```

output for `x: int = 3`

```
%x.addr = alloca i32
store i32 3, ptr %x.addr
```

An initializer is a **literal** — a number or `True/False` written out — never an expression. The grammar says so: `var_def ::= typed_var = literal NEWLINE` (ChocoPy §4). So `val` is always a constant here.



out.env

```
{ x → %x.addr }
```

Reading **x** is one instruction: fetch it.

```
case ast.Name(id=x):
    t = out.fresh()
    out.emit(f"{t} = load i32, ptr {out.env[x]}")
    return t
```

the puzzle's $x * (x + 4)$ · same shape as Wednesday's slide 18

```
compile_expr( x * (x + 4) )
| a = compile_expr( x )      emit  %t1 = load i32, ptr %x.addr  → "%t1"
| b = compile_expr( x + 4 )
| | a = compile_expr( x )    emit  %t2 = load i32, ptr %x.addr  → "%t2"
| | b = compile_expr( 4 )    → "4"
| | emit  %t3 = add i32 %t2, 4  → "%t3"
| emit  %t4 = mul i32 %t1, %t3  → "%t4"
```

Wednesday's recursion, untouched. The only difference: a leaf `x` now emits a `load`, where a leaf `3` emitted nothing.

The puzzle, answered.

var.py · prints 21

```
x: int = 3
print(x * (x + 4))
```

```
%x.addr = alloca i32           ; slide 8
store i32 3, ptr %x.addr       ; slide 8
%t1 = load i32, ptr %x.addr    ; slide 9
%t2 = load i32, ptr %x.addr    ; slide 9
%t3 = add i32 %t2, 4
%t4 = mul i32 %t1, %t3
call void @print_int(i32 %t4)
```



Same answer as Wednesday, 21. What changed is where the 3 lives: not inside an instruction, but in a slot, fetched each time it's needed. Slide 3's first assumption, repaired.

This is what clang does too.

clang -O0 -S -emit-llvm c1.c · align 4 and nsw removed

```
int f(void) {  
    int x = 3;  
    return x * (x + 4);  
}
```

```
%1 = alloca i32  
store i32 3, ptr %1  
%2 = load i32, ptr %1  
%3 = load i32, ptr %1  
%4 = add i32 %3, 4  
%5 = mul i32 %2, %4  
ret i32 %5
```

```
; %x.addr = alloca i32  
; store i32 3, ptr %x.addr  
; %t1 = load i32, ptr %x.addr  
; %t2 = load i32, ptr %x.addr  
; %t3 = add i32 %t2, 4  
; %t4 = mul i32 %t1, %t3
```

The same program in C, through clang: **line for line what we emitted on slide 10**, down to the two loads. Only the names differ — clang numbers, we name.

Assignment: a store.

```
case ast.Assign(targets=[ast.Name(id=x)], value=v):    # x = e
    val = compile_expr(v, out)
    out.emit(f"store i32 {val}, ptr {out.env[x]}")
```

output for `x = x + 1`, with `x` holding 3

```
%t1 = load i32, ptr %x.addr    ; read x          (slide 9)
%t2 = add i32 %t1, 1
store i32 %t2, ptr %x.addr    ; write x          (this slide)
```



Two rules, and that is the whole story of variables: **read = load**, **write = store**. The slot outlives every expression that touches it.

True is 1. For now.

```
case ast.Constant(value=bool(b)):      # MUST come before the int case:
    return "1" if b else "0"          # in Python, True is an int
case ast.Constant(value=int(n)):        # Wednesday's case, now second
    return str(n)
```

We represent booleans as `i32` so that every value has the same type — so `compile_expr` has one kind of thing to return, and not one case in it ever has to ask “what type is this?”

That is a **trade**, and this is the only place it gets made: slightly redundant IR, in exchange for a compiler with no type bookkeeping inside it. You will see the price charged twice — slides 15 and 17 — and then sitting in every worked example after that. It is the price, not a mistake. A real checker (week 5) makes `print(True)` print `True`; today it prints `1`.

Comparisons produce a different type.

```
CMPOPS = {ast.Eq: "eq", ast.NotEq: "ne", ast.Lt: "slt",
          ast.LtE: "sle", ast.Gt: "sgt", ast.GtE: "sge"}

case ast.Compare(left=l, ops=[op], comparators=[r]):
    a = compile_expr(l, out); b = compile_expr(r, out)
    t1 = out.fresh()
    out.emit(f"{t1} = icmp {CMPOPS[type(op)]} i32 {a}, {b}")    # result is i1
    ...
```

i1 a one-bit value — LLVM's true/false, a different type from i32

s in slt signed: -1 is less than 1. There is an unsigned family too, and we'll want it in week 7.

ops=[op] exactly one comparison. ChocoPy has no chains (§4.1) — but Python's parser hands us `a < b < c` happily. This is where we refuse it.

`icmp` compares two i32s and hands back an i1. Types are not a suggestion in LLVM.

...so we widen it back.

```
t2 = out.fresh()
out.emit(f"{t2} = zext i1 {t1} to i32")           # widen to i32
return t2
```

output for the running condition $x > 5$, with x in its slot

```
%t1 = load i32, ptr %x.addr
%t2 = icmp sgt i32 %t1, 5           ; i1 (slide 14)
%t3 = zext i1 %t2 to i32           ; i32 (this slide)
```

zext pads a narrow value with zeros to make a wider one. Slide 13's trade, first instalment: one type everywhere, no special cases in `compile_expr`.

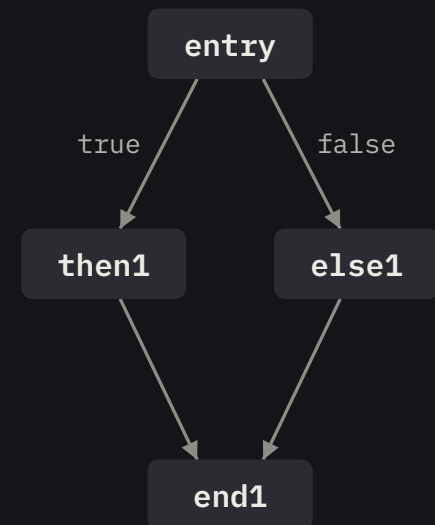
if is not an instruction. It never was.

```
x: int = 3
if x > 5:
    print(1)
else:
    print(0)
```

Wednesday, slide 9: the machine runs one instruction at a time, in order, and the only thing that ever changes that order is a jump to a label. Nothing has been added since.

So **if** cannot survive the trip down, and neither can **while**. Wednesday slide 10 said LLVM IR is C with only **goto** — this is the slide where that starts to cost you something.

Today's job: write by hand the **gotos** a C programmer would write if someone took **if** away from them.

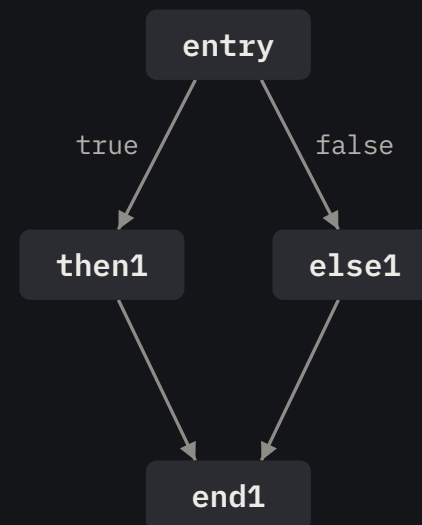


Slide 3's second assumption, dying: the order of the text is no longer the order of execution.

Labels and jumps, in LLVM.

```
entry:
  ...compute the condition into %c (an i1)...
  br i1 %c, label %then1, label %else1
then1:
  call void @print_int(i32 1)
  br label %end1
else1:
  call void @print_int(i32 0)
  br label %end1
end1:
  ret i32 0
```

```
def to_i1(v, out):
    """Our values are i32; br needs an i1."""
    t = out.fresh()
    out.emit(f"{t} = icmp ne i32 {v}, 0")
    return t
```



condition

the value an if or a while tests in order to decide what runs next — here, %c

```
br i1 %c, label %a, label %b
if (c) goto a; else goto b;
```

```
br label %a
goto a;
```

No falling through.

Every labeled chunk must end in a jump or a `ret`. Not the machine's doing: in C a labelled statement falls through into the next. It is LLVM's own bookkeeping rule — and the PA1 mistake you will actually make.

Slide 15 widened the comparison to `i32`; `br` wants one bit back. `!= 0` recovers it — slide 13's trade, second and last instalment.

Compiling **if**.

joining the Emitter now – if is the first thing that needs them

```
self.k = 0                                # in __init__: counter for label groups
def fresh_id(self):                       # one number shared by an if's three labels
    self.k += 1
    return self.k
def label(self, name):                   # write "name:" with no indent
    self.lines.append(name + ":")
```

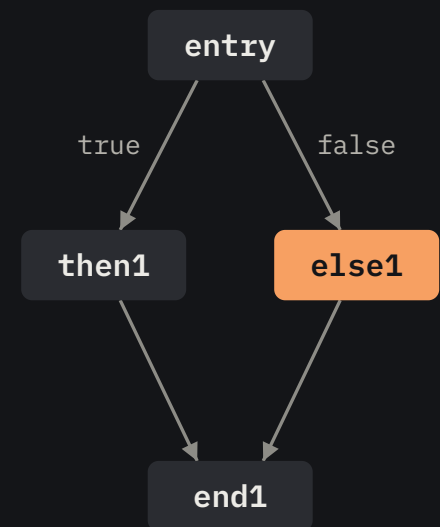
```
case ast.If(test=c, body=then, orelse=els):
    n = out.fresh_id()                   # one number for this if's labels
    cond = to_i1(compile_expr(c, out), out)
    out.emit(f"br i1 {cond}, label %then{n}, label %else{n}")
    out.label(f"then{n}")
    for st in then: compile_stmt(st, out)
    out.emit(f"br label %end{n}")
    out.label(f"else{n}")
    for st in els: compile_stmt(st, out)
    out.emit(f"br label %end{n}")
    out.label(f"end{n}")
```

Why the fresh number? Two **ifs** in one program can't share a label name.

Worked example: the whole **if**.

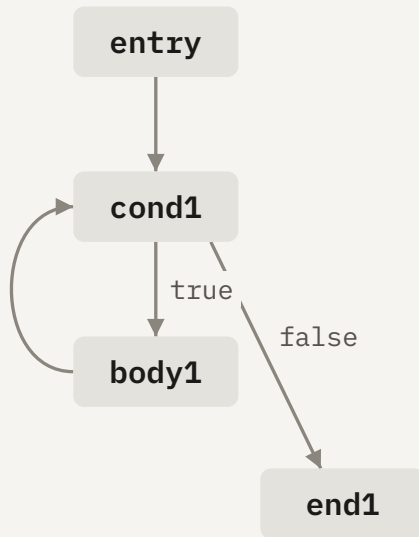
cocoa3.py ifx.py · the slide-16 program · prints 0 · trimmed: end1 goes on to
ret i32 0

```
entry:
  %x.addr = alloca i32
  store i32 3, ptr %x.addr
  %t1 = load i32, ptr %x.addr
  %t2 = icmp sgt i32 %t1, 5           ; an i1
  %t3 = zext i1 %t2 to i32           ; widened   (slide 15)
  %t4 = icmp ne i32 %t3, 0           ; narrowed (slide 17)
  br i1 %t4, label %then1, label %else1
then1:
  call void @print_int(i32 1)
  br label %end1
else1:
  call void @print_int(i32 0)
  br label %end1
end1:
```



Every line came from a slide you've already seen: the first two from slide 8, the next three from slides 14–15, the `br` from slide 17, the chunks from slide 18. `x` is 3, so `%t4` is false and control goes to **else1**: it prints 0.

while: the same trick, plus a jump backwards.



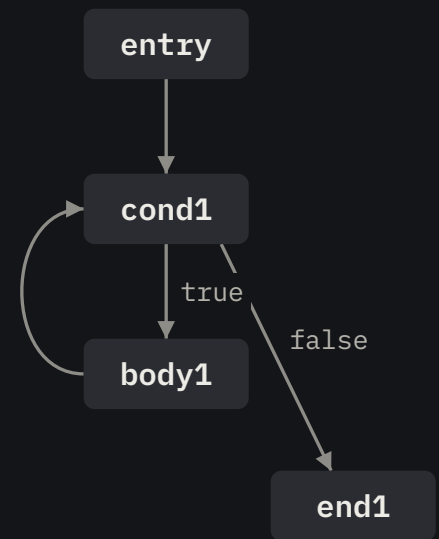
Test the condition in its own chunk.

The body jumps *back* to the test.

That backward jump is the loop — there is nothing else to it.

Compiling **while**.

```
case ast.While(test=c, body=body):
    n = out.fresh_id()
    out.emit(f"br label %cond{n}")
    out.label(f"cond{n}")
    cond = to_i1(compile_expr(c, out), out)
    out.emit(f"br i1 {cond}, label %body{n}, label %end{n}")
    out.label(f"body{n}")
    for st in body: compile_stmt(st, out)
    out.emit(f"br label %cond{n}")
    out.label(f"end{n}")
```



Notice the first line: we jump to cond even though cond is the very next thing. No falling through.

Worked example: sum 1..n.

```
3 alloca/store pairs trimmed · end1 → ret
i32 0
```

sumto.py · prints 6

```
n: int = 3
i: int = 1
s: int = 0
while i <= n:
    s = s + i
    i = i + 1
print(s)
```

```
cond1:
    %t1 = load i32, ptr %i.addr
    %t2 = load i32, ptr %n.addr
    %t3 = icmp sle i32 %t1, %t2
    %t4 = zext i1 %t3 to i32
    %t5 = icmp ne i32 %t4, 0
    br i1 %t5, label %body1, label %end1

body1:
    %t6 = load i32, ptr %s.addr
    %t7 = load i32, ptr %i.addr
    %t8 = add i32 %t6, %t7
    store i32 %t8, ptr %s.addr
    %t9 = load i32, ptr %i.addr
    %t10 = add i32 %t9, 1
    store i32 %t10, ptr %i.addr
    br label %cond1

end1:
    %t11 = load i32, ptr %s.addr
    call void @print_int(i32 %t11)
```

the slots, each arrival at cond1

visit	n	i	s	i <= n ?
1st	3	1	0	yes → body1
2nd	3	2	1	yes → body1
3rd	3	3	3	yes → body1
4th	3	4	6	no → end1, print 6

Four visits to cond1, three to body1, one line of source run three times. i and s are places; each time round: load, compute, store.

And look at what this one program is: slide 3's two assumptions, both broken and both repaired. Values that outlive the expressions that made them, in slots. An order of execution the text no longer shows you, in labels.

From 68 lines to 129.

diff cocoa2.py cocoa3.py · + added · unchanged in grey

```
class Emitter:
    def __init__(self): lines, n
+     self.env = {}
+     self.k = 0
    def fresh(self)
+     def fresh_id(self)
+     def label(self, name)

    BINOPS = {...}
+    CMPOPS = {...}
+    def to_i1(v, out)

    def compile_program(src)
```

```
def compile_expr(e, out):
+     case Constant(bool)      # above int
    case Constant(int)
+     case Name
    case BinOp
    case UnaryOp
+     case Compare

def compile_stmt(s, out):
    case print(...)
+     case AnnAssign
+     case Assign
+     case If
+     case While
```

Incremental. The language grows; the old code stays.

Live · 1 of 2 · about 60 seconds

Run the loop.

```
$ python3 cocoa3.py sumto.py > sumto.ll && clang sumto.ll runtime.c -o sumto && ./sumto
```

6

The number slide 22's trace table predicted — you computed it before the machine did.

Live · 2 of 2 · two minutes

Break it.

broken.ll = slide 19's if program, with then1's br label %end1 deleted

```
$ clang broken.ll runtime.c  
broken.ll:14:1: error: expected instruction opcode
```

clang checks *our* output. Every chunk must end in a jump. This is the first time a tool has told you your compiler is wrong. Wednesday: a tool that tells you when it's wrong *and still compiles*.

We emit far more loads and stores than we need.

our body1 (slide 22) · 7 instructions · 3 loads · 2 stores

```
body1:
    %t6 = load i32, ptr %s.addr
    %t7 = load i32, ptr %i.addr
    %t8 = add i32 %t6, %t7
    store i32 %t8, ptr %s.addr
    %t9 = load i32, ptr %i.addr
    %t10 = add i32 %t9, 1
    store i32 %t10, ptr %i.addr
    br label %cond1
```

clang, at -O1, keeps i and s in the machine's hands for the whole loop — Wednesday slide 9's third fact — and never touches the memory strip at all. **Zero loads, zero stores.**

In week 8 you'll learn what it did, and do it yourself. Until then: correctness first.

Wednesday: testing your compiler.

Testing your compiler — and first, why a compiler is harder to test than anything you've tested before.

Bring the `//` question from last Wednesday; you'll find the bug without me.

PA1 due Friday; Thursday's section is a PA1 lab.

